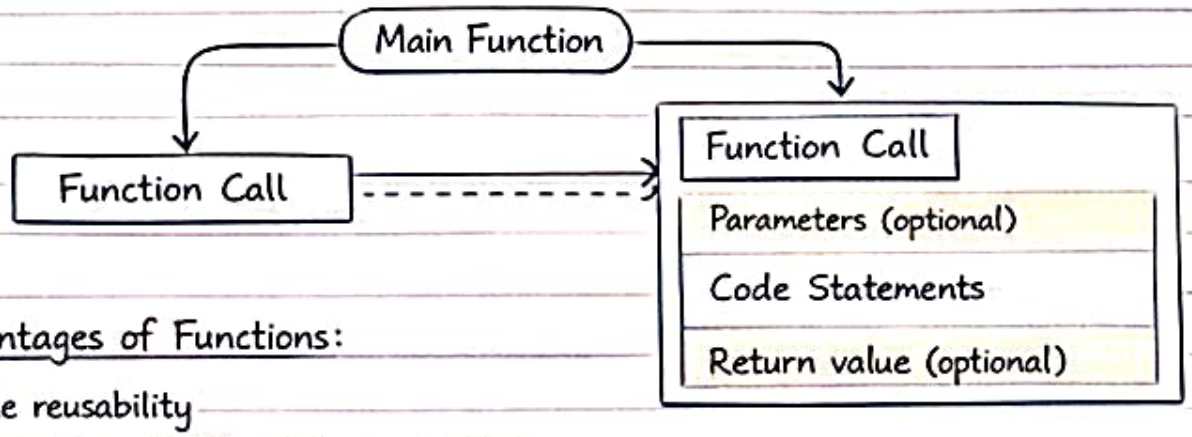
Functions in C Programming

Function Definition:

- A function is a block of code that performs a specific task and can be reused.

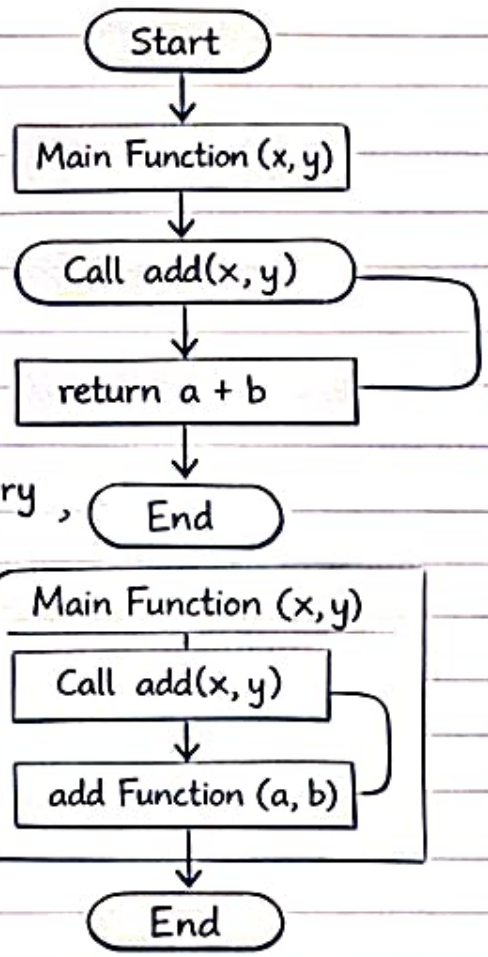


★ Advantages of Functions:

- Code reusability
- Improves readability and organization
- Easy debugging and maintenance.

Syntax:

```
return_type function_name(parameters)
{
  // code statements
  return (value);
}
```



Example:

```
#include <stdio.h>

int add(int a, int b)
 {
   return a + b;
 }
 int main()
 {  int x = 5, y = 3;
    int result = add(x, y);
    printf("Sum: %d\n", result);
 }
```

Output: Sum: 8

Arrays in C Programming

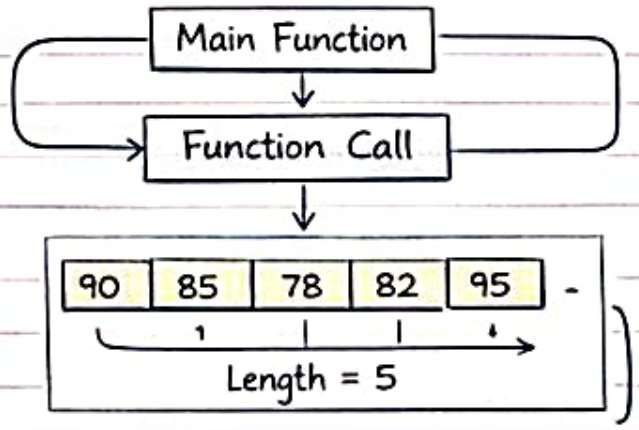
Definition:

- An array is a collection of variables of the same data type, stored in contiguous memory locations.

Declaration:

- Syntax:

```
data_type array_name[size];
```

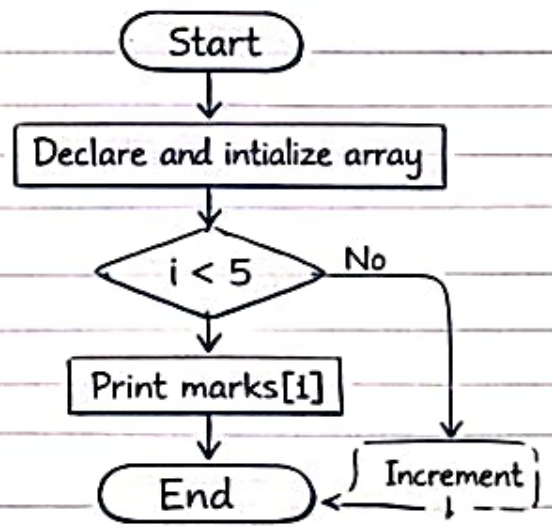


Example:

```
int marks[5]; // declares an array of 5
    integers named "marks".
```

Initializing an Array:

- We can assign values to the array elements like this:
  - marks[0] = 90;
  - int marks[5] = {90, 85, 78, 82, 95};



Accessing Array Elements:

- Syntax to print an element:

```
printf("%d", array_name[index]);
```

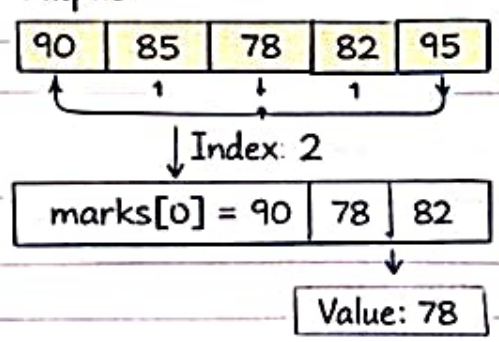
Example Program:

```
#include <stdio.h>

int main()
 {
    int marks[5] = {90, 85, 78, 82, 95};
    int i;

    for(i = 0; i < 5; i++)
    {
        printf("marks[%d] = %d\n", marks[i]);
    }
 }
```

Output:



Output:

```
marks[0] = 90
marks[1] = 85
marks[2] = 78
marks[3] = 82
marks[4] = 95
```

Example Program:

Dynamic Memory Allocation in C

What is Dynamic Memory Allocation?

- Dynamic memory allocation allows us to allocate memory at runtime (while the program is running) using pointers.

Heap Memory

variable	address	value
a	1000	10
*ptr	3000	10

3200 3100

time

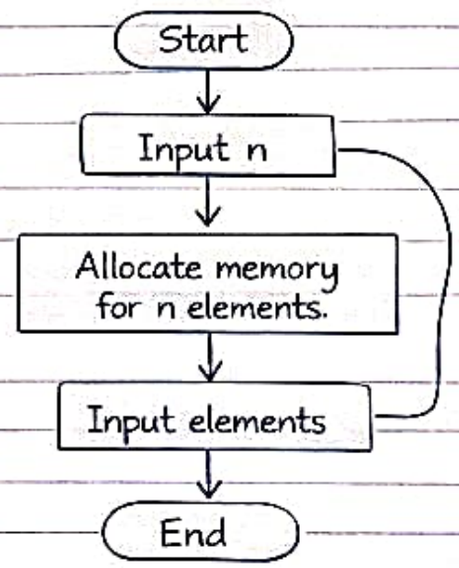
Functions for Memory Allocation:

* malloc(size): Allocates a specified number of bytes and returns a pointer to the first byte
* calloc(num_elements, size): Allocates memory for an array of elements, initializes them to zero.
* free(ptr): Deallocates the memory previously allocated by malloc, calloc or realloc.
* realloc(ptr, new_size): Resizes previously allocated memory block to the new size.

Example Program:

```
#include <stdio.h>
#include <stdlb.h>

int main()
{
    int n, i;
    int *ptr;
    printf("Enter number of elements:*");
    scanf("%d", &n);
}
ptr = (int*) malloc(n * sizeof(int));
if (ptr = NULL) {
    printf("Memory allocation failed!\n*);
    return 1;
}
```



Example Program:

```
Enter number of elements: 5
Enter 5 integers:
1  2.  3  4.  5
The elements are: 1 2 3 4 5
```

Output:

- Enter number of elements: 5
- Enter 1 integers:
- Element 0: 10
- Element 2: 20
- Element 3: 30
- Element 4: 50

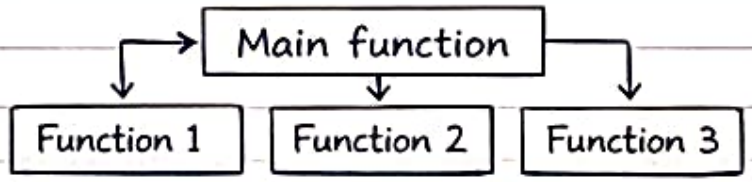
@mastercode.sagar

Functions in C

Definition:

- A function is a block of code thet performs a specific task. It helps to divide a program into smaller, manageable parts.

* Functions:

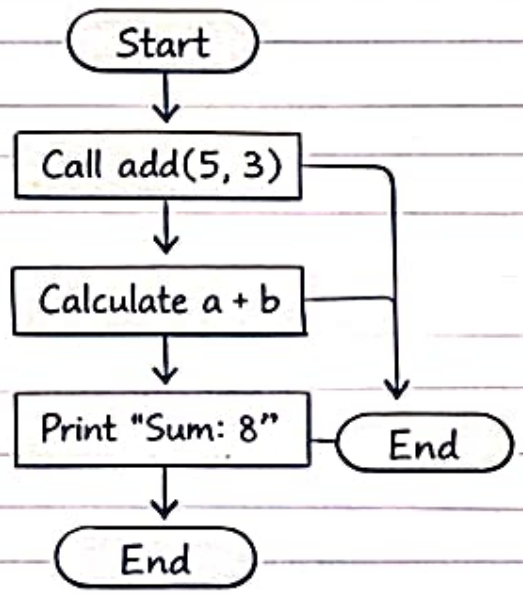


Advantages:

- Code reusability: Functions allow us to reuse code.
- Modularity: Functions make programs easier to understand and maintain.
- Debugging: Easier to debug and test smaller parts of the program.

Syntax & Example Program:

```
return_type function_name(parameters)
{  // Function body
   return value;
}
```




```
#include <stdio.h>
//- Function declaration
int add(int a,int b)
int main()
{
   int sum;
   sum = add(5,3);
   printf("Sum: %dn", sum);
}
```

* Function definition

```
int add(int a, int b)
{
   return a + b;
}
```

Output: Sum : 8

Control Statements in C

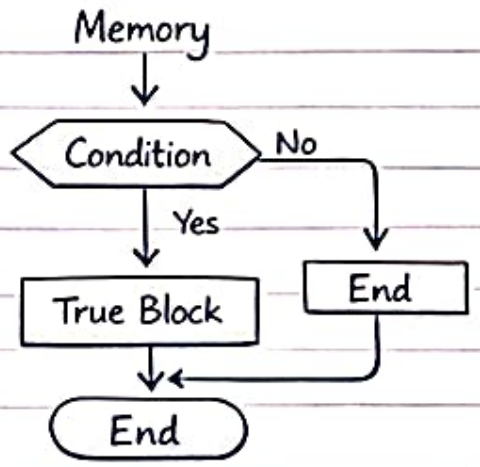
Conditional Statements:

Conditional statements are used to make decisions in a program based on certain conditions.

★ **if statement:**

- Executes a block of code if a condition is true.

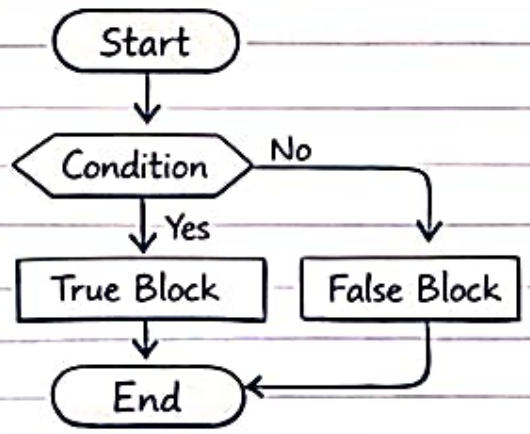
```
Syntax:
if(condition)
 {
   // code to execute if condition is true
 }
```



★ **if-else statement:**

- Executes one block of code if the condition is true, and another block if the condition is false.

```
Syntax:
if(condition)
 {
   // code to execute if condition is false.
 else
 {
   // code to execute if condition is false.
 }
```

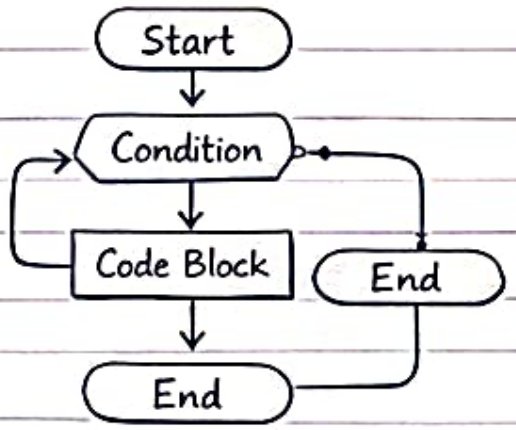


Loop Statements:

- Loop statements are used to execute a block of code repeatedly.

★ **while loop:**

```
while(condition)
 {
   // code to be executed while
   condition is true
 }
```



★ **for loop:**

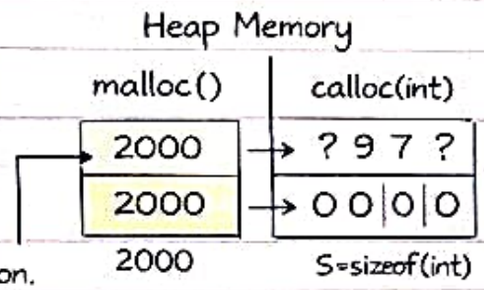
- Used when the number of iterations is known.

@mastercode.sager

Dynamic Memory Allocation in C Programming

Pros and Cons of Dynamic Memory Allocation:

Pros:



* Efficient use of memory allocate memory as needed.
* Helps develop flexible and generic data structures (e.g. linked lists, trees).
* Can handle varying data sizes during program execution.
* More complex code due to manual memory management.
* Risk of memory leaks if allocated memory is not properly freed.
* Slower allocation/deallocation compared to static memory allocation.

Common Mistakes to Avoid:

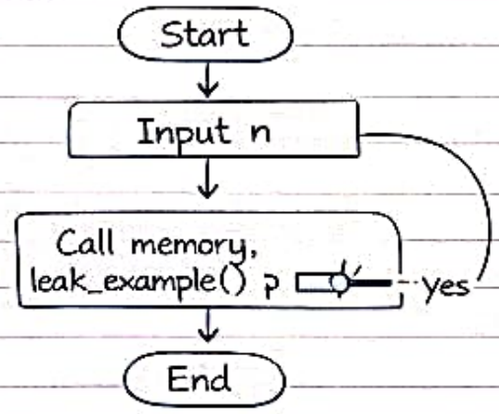
- Forgetting to free dynamically allocated memory, leading to memory leaks.
* Dereferencing a NULL pointer if memory allocation fails.
* Accessing memory after it has been freed, resulting in undefined behavior.
* Not checking if memory allocation (malloc, calloc, realloc) was successful.

Memory Leak Example:

```c
#include <stdio.h>
#include <stdlb.h>

void memory_leak_example()
{ int *ptr;
  ptr = (int*) malloc(10 * sizeof(int));
} scanf("%d", &n);

int main()
while (1)
{
  memory_leak_example();
  }
return 0;
}
```



This continuous process leads to a memory leak because the allocated memory is never freed.

Output:

- Enter initutal number of elements: 5
- Enter 5 elements: 1 2 3 4 5
- Enter new size for the array: 8
- The elements are: 1 2 3 4 5 6 6 7 8

Best Practices:

- Enter initial number of elements: 5
- Enter 5 elements: 1 2 3 4 5
- Enter new size for the array: 8
- Enter 3 more elements: 6 7 8
- The elements are: 1 2 3 4 5 5 6 7 8

@mastercode.sagar

Introduction to C Language

C is a powerful and popular programming language used for system and application development.

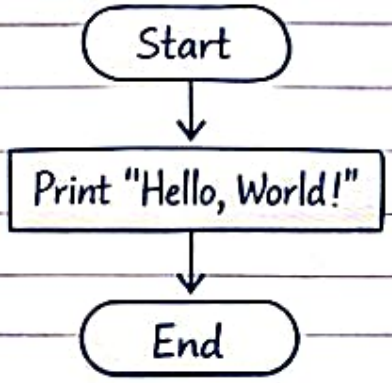
Features of C:

- Simple and efficient
- Fast execution
- Low-level access to memory
- Portable and versatile

Syntax:

```
#include <stdio.h>

int main()
 {
    // code statements
    return 0;
 }
```



Example Program:

```
#include <stdio.h>

int main()
 {
    printf("Hello, World!"n");
    return 0;
 }
```

Output: Hello, World!

Basics of C Programming

Variables & Data Types:

Variables are used to store data. In C, each variable must have a specific data type.

Common data types:

- int : Integer (stores whole numbers)
- float : Floating-point number (stores decimal numbers)
- char : Character (stores a single character)
- double : Double-precision floating-point number

Memory

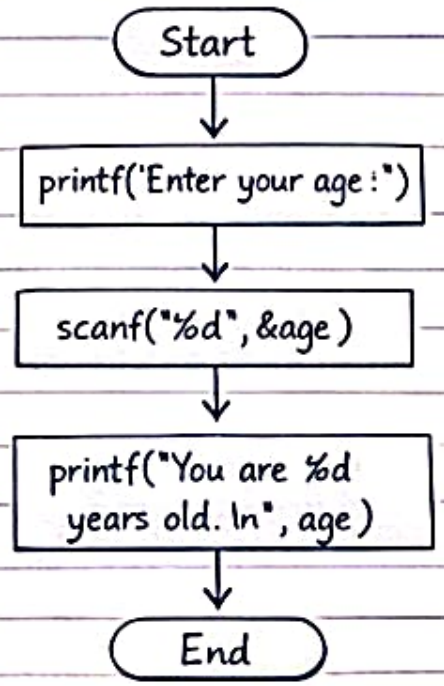
int a	10
char b	'A'
float c	3.14
double d	2.71828

Input & Output Functions:

- printf() : Used to print output to the console.
- scanf() : Used to take input from the user.

Example:

```c
#include <stdio.h>
int main()
{
    int age;
    printf("Enter your age: ");
    scanf("%d", &age);
    printf("You are %d years old.\n", age);
    return 0;
}
```



Input: Enter your age: 22

Output: You are 22 years old.

Pointers in C

What is a Pointer?

- A pointer is a variable that stores the address of another variable.

Why use Pointers?

- Efficient memory management
- Implement data structures like linked lists, trees, etc.
- Allows pass-by-reference, enabling functions to modify variables directly

Memory

variable	address	value
a	1000	→ 100
*ptr	2000	1000

Pointer Syntax:

```
data_type *pointer_name;
↳ data_type> holds type of the variable
```

Example:

```
int a = 10;
int *ptr;
```

Memory

Main Function (x, y)	
1000 →	10
2000	1000

Memory

Output:

```
int a = 10;
int *ptr;
ptr = &a; // ptr now points to the address of 'a'
printf("Value of a: %d\n", a); Output: 10 (value of a)
printf("Value of *ptr: %d\n", *ptr); Output: 10 (value of *ptr)
printf("Address of a: %p\n", &a); Output: 1000 (address of a)
printf("Address stored in ptr: %p\n", ptr); Output: 1000 (addres of a)
```

variable	address	value
a	1000 →	10
*ptr	2000	1000

- Value of a: 10
- Value of *ptr: 10
- Address of a: 1000
- Address stored in ptr: 1000

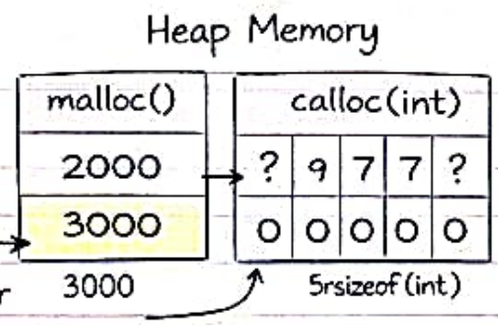
@mastercode.sagar

Dynamic Memory Allocation in C Programming

Memory Allocation Functions:

* malloc(size): Allocates a specified number of bytes and returns a pointer to the first byte.
  ↳ Memory is uninitialized (garbage values).

* calloc(num_elements, size): Allocates memory for an array of elements, initializes them to zero, and returns a pointer



* free(ptr): Deallocates the memory previously allocated by malloc, calloc or realloc.

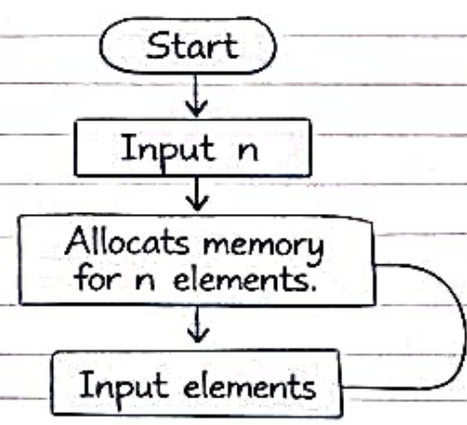
* realloc(ptr, new_size): Resizes previously allocated memory block to the new size.

Differences between malloc() and calloc():

Initalization	malloc()	calloc()	realloc()
Initaxon	• Memory is	5 * *sizeof(int)	5 * *sizeof(int)
Syntax:	• Memory is initialized to zero	ptr = (int*)(nalloc)	ptr = (int*).Jall(in),
Usage:	• Allocates a single block of memory.	single 6tek of memory initializes them	Allocates multiple blocks of memory and initializes them to zero.

Example with realloc():

```
#include <stdio.h>
#include <stdlb.h>
int main()
{
   int *ptr;
   int n, m, i;
   printf("Enter initial number of elements: ");
   scanf("%d", &n);
 }
 ptr = (int*) malloc(n * sizeof(int));
 printf("Enter %a elements."\n', n);
 for (: i < m, ; ++)
  {
    printf("%d", ptr[i]);
  }
  free(ptr);
}
```



Example Program:

```
Enter initial number of elements: 5
Enter 5 elements: 1 2 3 4 5
Enter new size for the array: 3
Enter 3 more elements: 6 7 5
The elements are: 1 2 3 4 5 6 7 8
```